LECTURE 8

CEN 348-Artificial Intelligence Systems

Computation graphs

A computation graph is a representation of the process of computing a mathematical expression, in which the computation is broken down into separate operations, each of which is modeled as a node in a graph.

Consider computing the function $L(a,b,c) = c(a+2b)$. If we make each of the component addition and multiplication operations explicit, and add names (d and e) for the intermediate outputs, the resulting series of computations is:

$$
\begin{aligned}
d &= 2*b \\
e &= a+d \\
L &= c*e
\end{aligned}
$$

Computation graphs

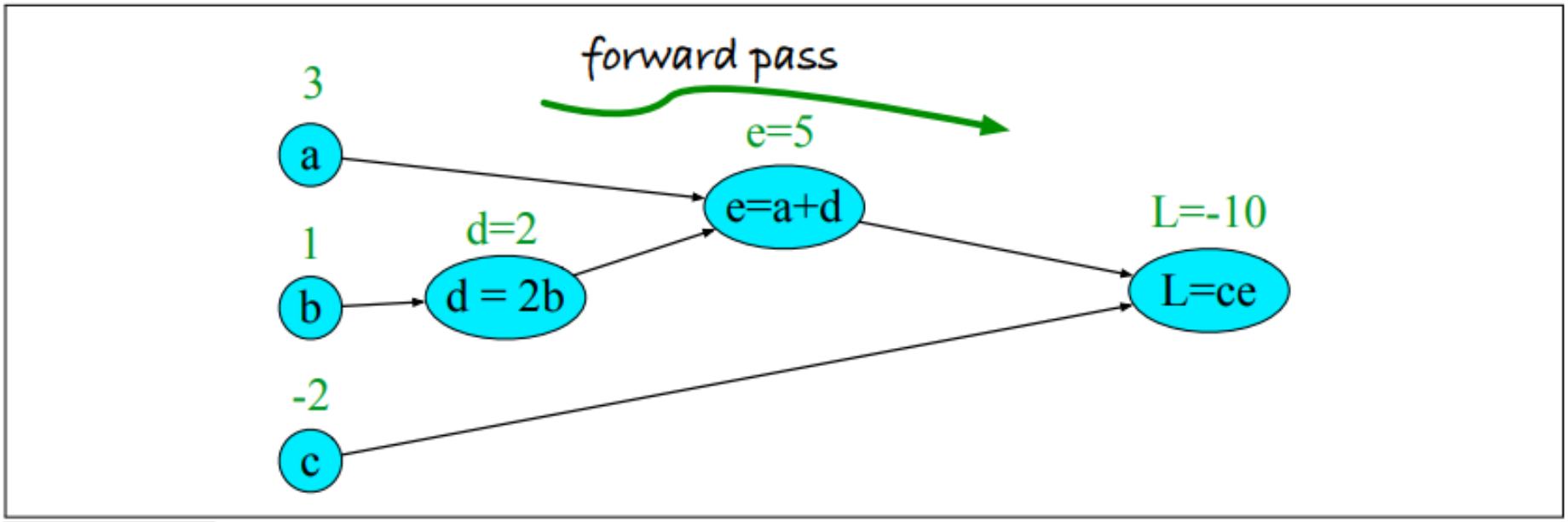


Figure 7.10 Computation graph for the function $L(a,b,c) = c(a+2b)$, with values for input nodes $a = 3$, $b = 1$, $c = -2$, showing the forward pass computation of L.

Computation graphs

We can now represent this as a graph, with nodes for each operation, and directed edges showing the outputs from each operation as the inputs to the next, as in Fig. 7.10. The simplest use of computation graphs is to compute the value of the function with some given inputs. In the figure, we've assumed the inputs $a = 3$, $b = 1$, $c = -2$, and we've shown the result of the **forward pass** to compute the result $L(3, 1, -2) = -10$. In the forward pass of a computation graph, we apply each operation left to right, passing the outputs of each computation as the input to the next node.

Backward differentiation on computation graphs

The importance of the computation graph comes from the **backward pass**, which is used to compute the derivatives that we'll need for the weight update. In this example our goal is to compute the derivative of the output function L with respect to each of the input variables, i.e., $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$. The derivative $\frac{\partial L}{\partial a}$, tells us how much a small change in a affects L.

chain rule

Backwards differentiation makes use of the **chain rule** in calculus. Suppose we are computing the derivative of a composite function $f(x) = u(v(x))$. The derivative of $f(x)$ is the derivative of $u(x)$ with respect to $v(x)$ times the derivative of $v(x)$ with respect to x:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \tag{7.23}$$

The chain rule extends to more than two functions. If computing the derivative of a composite function $f(x) = u(v(w(x)))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx} \tag{7.24}$$

Backward differentiation on computation graphs

Let's now compute the 3 derivatives we need. Since in the computation graph $L = ce$, we can directly compute the derivative $\frac{\partial L}{\partial c}$:

$$\frac{\partial L}{\partial c} = e \tag{7.25}$$

For the other two, we'll need to use the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b} \end{aligned} \tag{7.26}$$

Eq. 7.26 thus requires five intermediate derivatives: $\frac{\partial L}{\partial e}$, $\frac{\partial L}{\partial c}$, $\frac{\partial e}{\partial a}$, $\frac{\partial e}{\partial d}$, and $\frac{\partial d}{\partial b}$, which are as follows (making use of the fact that the derivative of a sum is the sum of the derivatives):

$$\begin{aligned} L = ce &: \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e \\ e = a + d &: \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1 \\ d = 2b &: \quad \frac{\partial d}{\partial b} = 2 \end{aligned}$$

Backward differentiation on computation graphs

In the backward pass, we compute each of these partials along each edge of the graph from right to left, multiplying the necessary partials to result in the final derivative we need. Thus we begin by annotating the final node with $\frac{\partial L}{\partial L} = 1$. Moving to the left, we then compute $\frac{\partial L}{\partial c}$ and $\frac{\partial L}{\partial e}$, and so on, until we have annotated the graph all the way to the input variables. The forward pass conveniently already will have computed the values of the forward intermediate variables we need (like d and e) to compute these derivatives. Fig. 7.11 shows the backward pass. At each node we need to compute the local partial derivative with respect to the parent, multiply it by the partial derivative that is being passed down from the parent, and then pass it to the child.

Backward differentiation on computation graphs

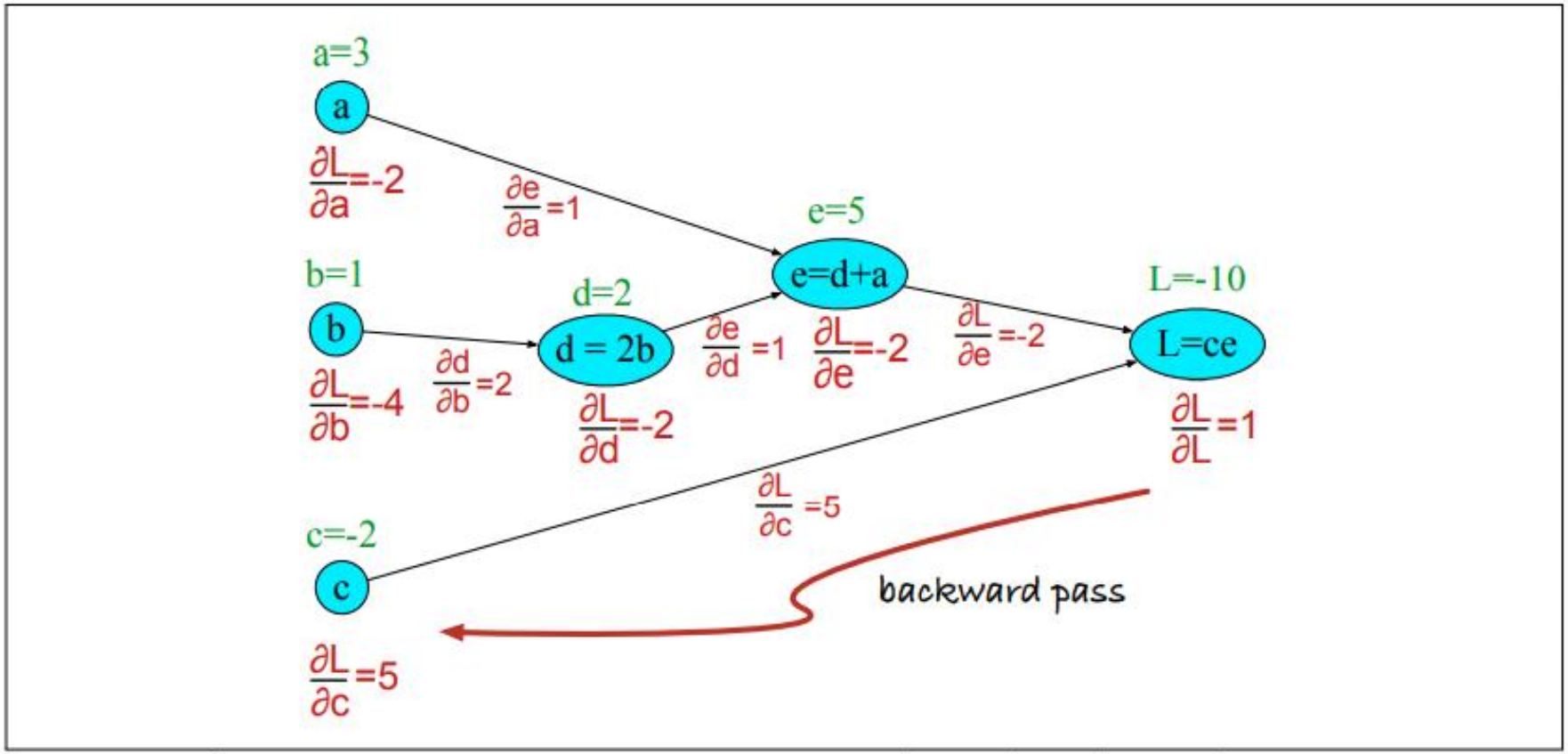


Figure 7.11 Computation graph for the function $L(a,b,c) = c(a+2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

Backward differentiation for a neural network

Of course computation graphs for real neural networks are much more complex. Fig. 7.12 shows a sample computation graph for a 2-layer neural network with $n_0 = 2$, $n_1 = 2$, and $n_2 = 1$, assuming binary classification and hence using a sigmoid output unit for simplicity. The function that the computation graph is computing is:

$$\begin{aligned} z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\ a^{[1]} &= \text{ReLU}(z^{[1]}) \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \\ \hat{y} &= a^{[2]} \end{aligned} \tag{7.27}$$

The weights that need updating (those for which we need to know the partial derivative of the loss function) are shown in orange. In order to do the backward pass, we'll need to know the derivatives of all the functions in the graph. We already saw in Section 5.8 the derivative of the sigmoid σ:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z)) \tag{7.28}$$

Backward differentiation for a neural network

We'll also need the derivatives of each of the other activation functions. The derivative of tanh is:

$$\frac{d\tanh(z)}{dz} = 1 - \tanh^2(z) \tag{7.29}$$

The derivative of the ReLU is

$$\frac{d\,\text{ReLU}(z)}{dz} = \begin{cases} 0 & for \quad x < 0 \\ 1 & for \quad x \geq 0 \end{cases} \tag{7.30}$$

Backward differentiation for a neural network

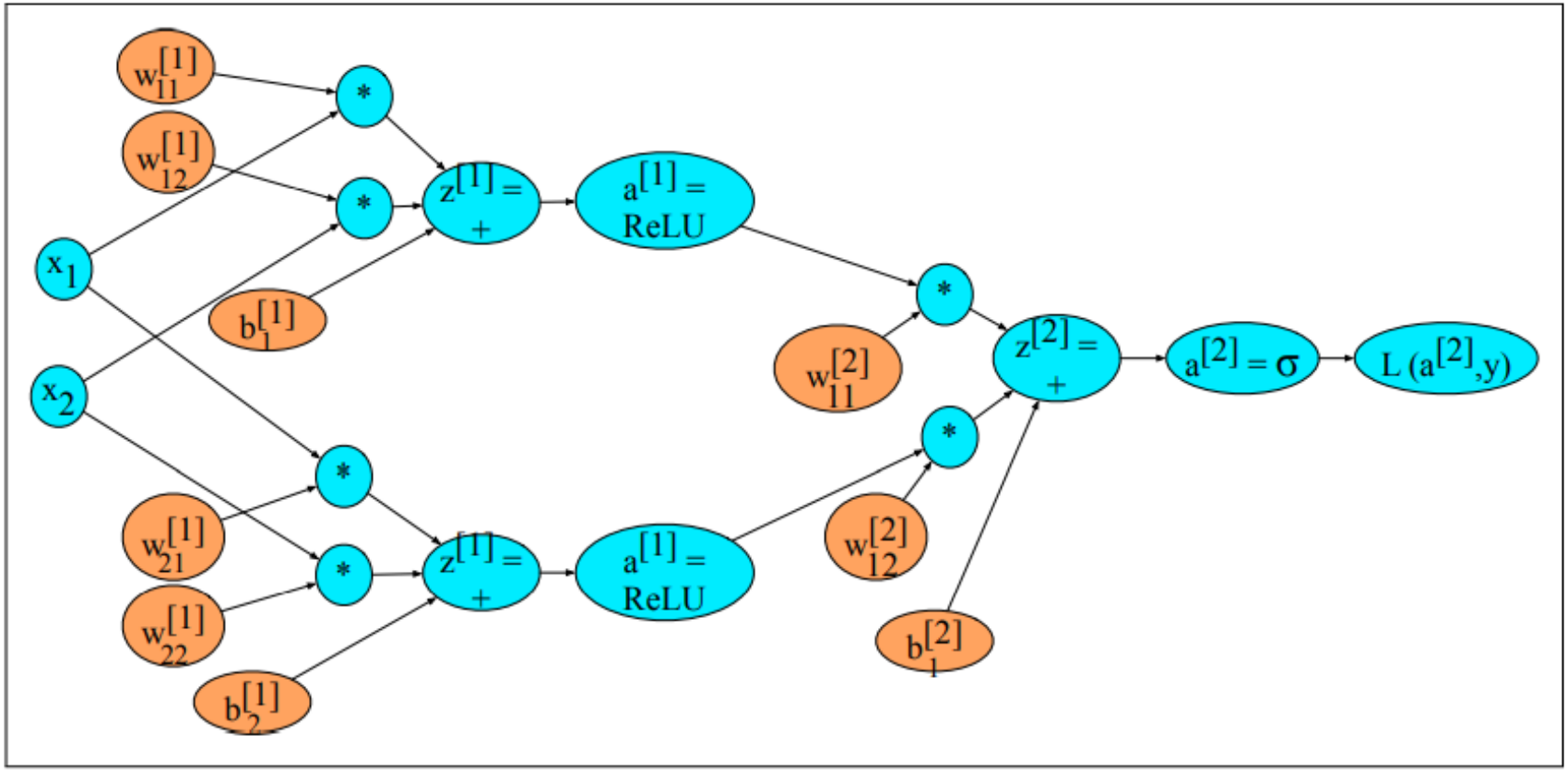
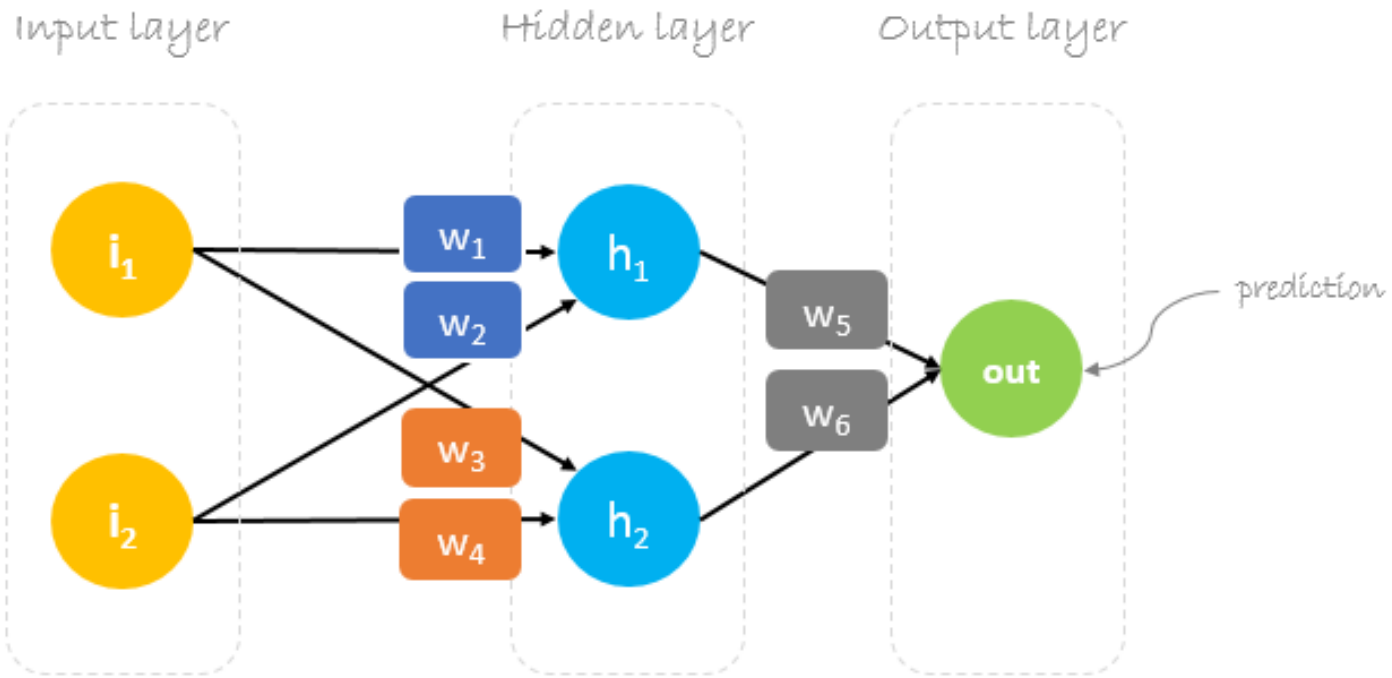


Figure 7.12 Sample computation graph for a simple 2-layer neural net (= 1 hidden layer) with two input dimensions and 2 hidden dimensions.

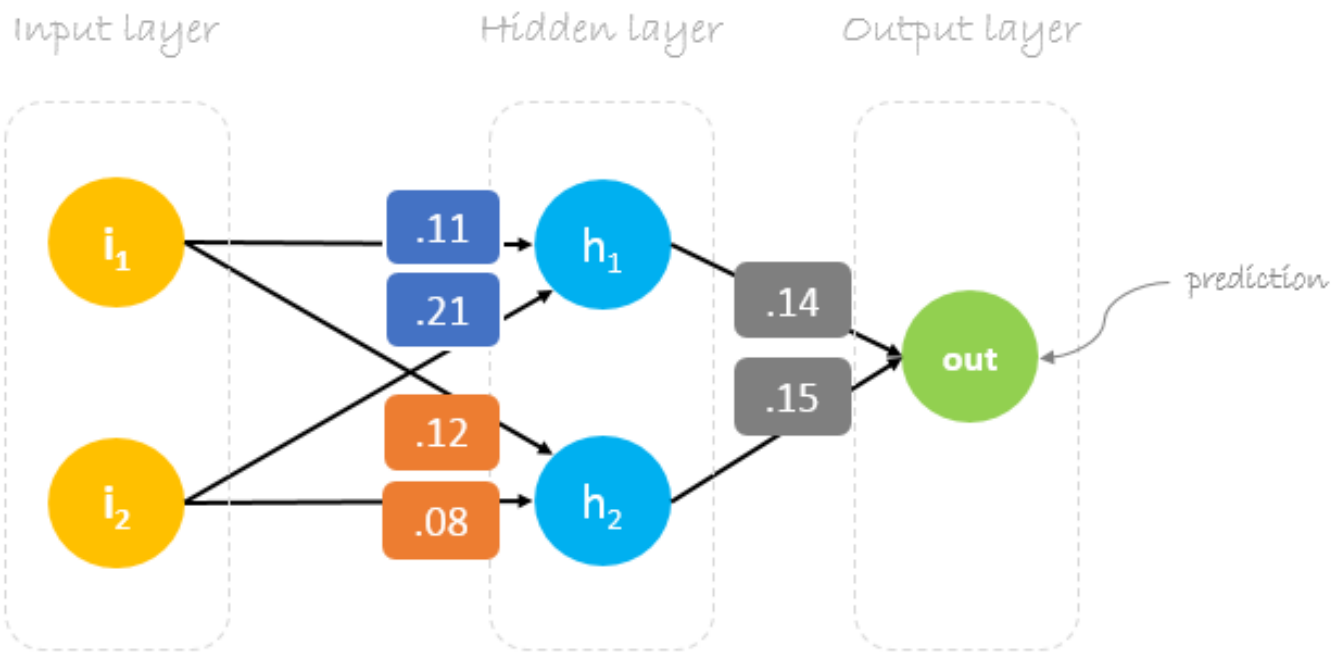
Backpropagation Step by Step: Example

We will build a neural network with three layers:

- **Input** layer with two inputs neurons
- One **hidden** layer with two neurons
- **Output** layer with a single neuron

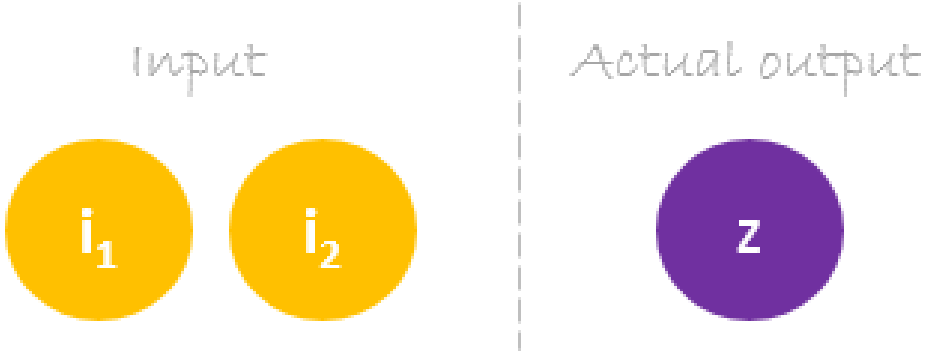


- Neural network training is about finding weights that minimize prediction error. We usually start our training with a set of randomly generated weights. Then, backpropagation is used to update the weights in an attempt to correctly map arbitrary inputs to outputs.
- Our initial weights will be as following: w1 = 0.11, w2 = 0.21, w3 = 0.12, w4 = 0.08, w5 = 0.14 and w6 = 0.15.



Dataset

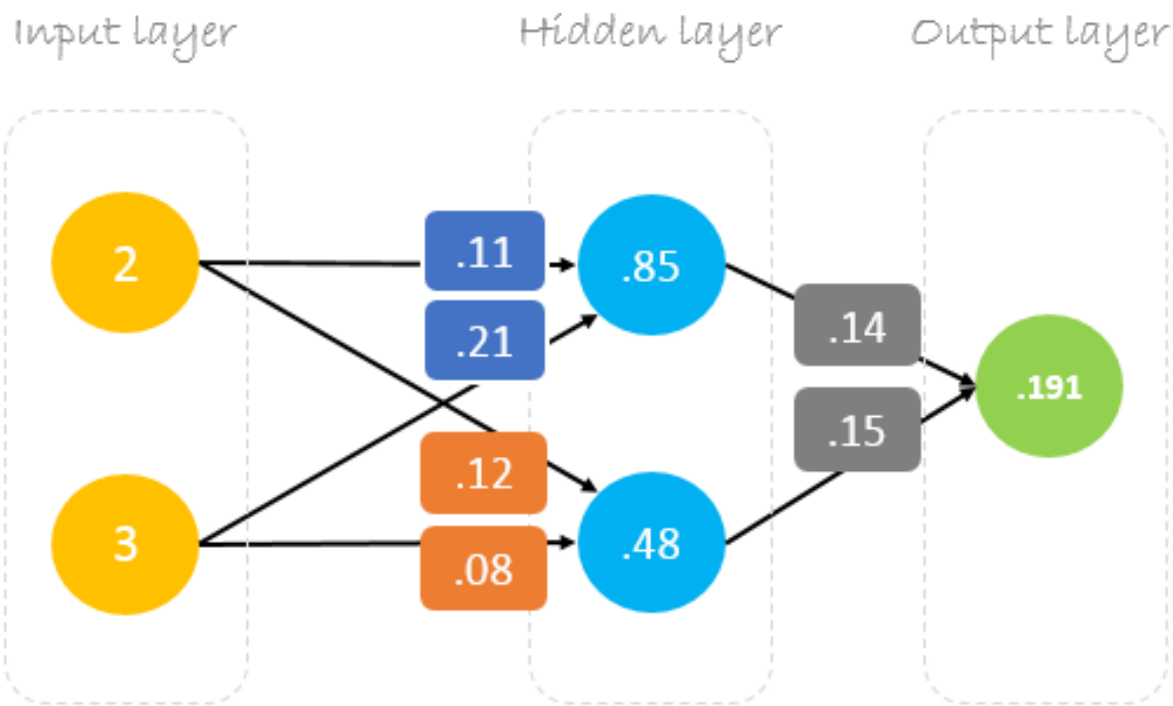
- Our dataset has one sample with two inputs and one output.



- Our single sample is as following inputs=[2,3] and output=[1].

Forward Pass

- We will use given weights and inputs to predict the output. Inputs are multiplied by weights; the results are then passed forward to next layer.



Matrix multiplication

$$[2 \quad 3] \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = [0.85 \quad 0.48] \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = [0.191]$$

Details

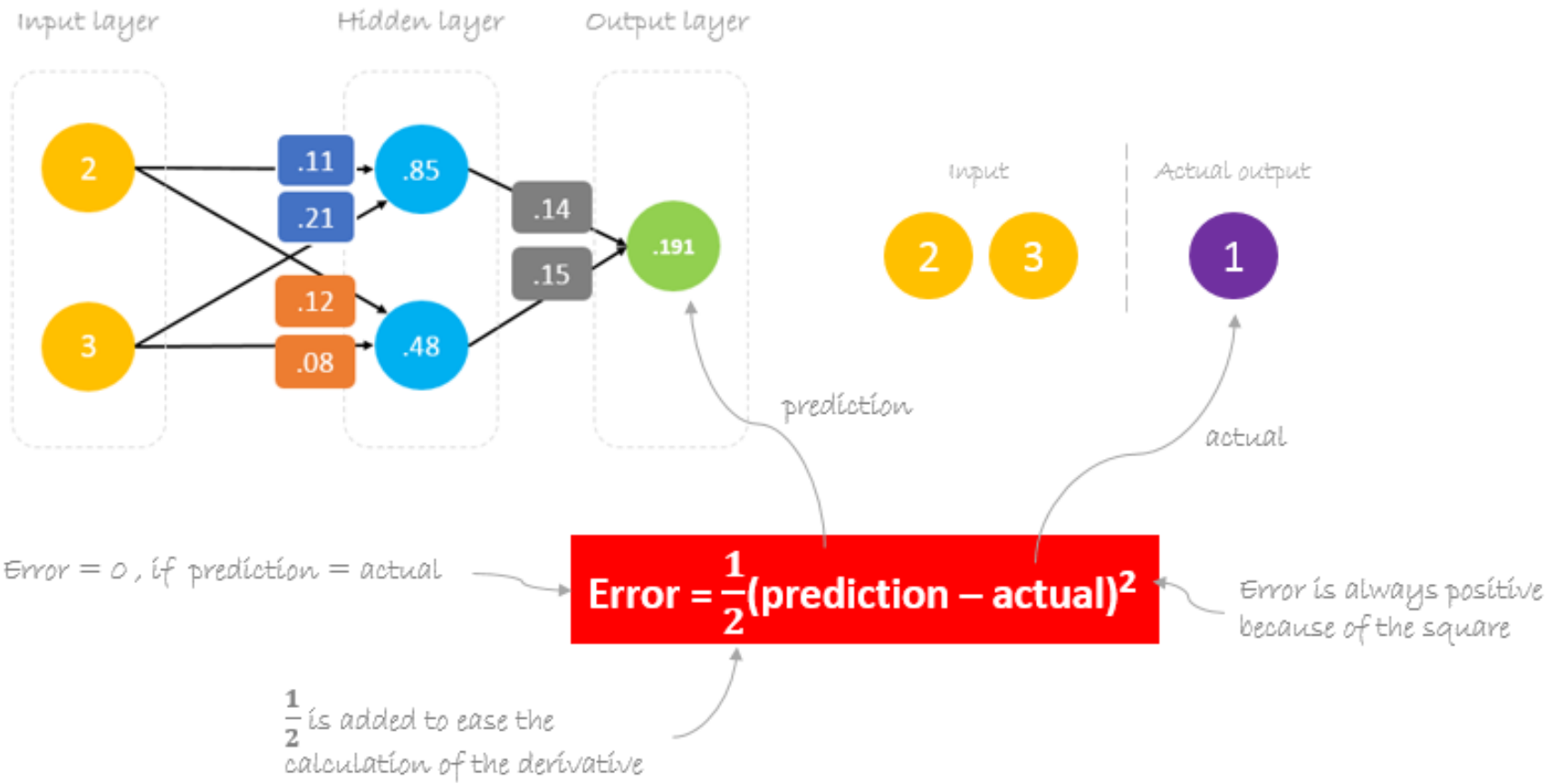
2 x .11 + 3 x .21 = .85

2 x .12 + 3 x .08 = .48

.85 x .14 + .48 x .15 = .191

Calculating Error

- Now, it's time to find out how our network performed by calculating the difference between the actual output and predicted one. It's clear that our network output, or prediction, is not even close to actual output. We can calculate the difference or the error as following.



Reducing Error

- Our main goal of the training is to reduce the **error** or the difference between **prediction** and **actual output**. Since **actual output** is constant, “not changing”, the only way to reduce the error is to change **prediction** value. The question now is, how to change **prediction** value?
- By decomposing **prediction** into its basic elements we can find that **weights** are the variable elements affecting **prediction** value. In other words, in order to change **prediction** value, we need to change **weights** values.

$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$h_2 = i_1 w_3 + i_2 w_4$$

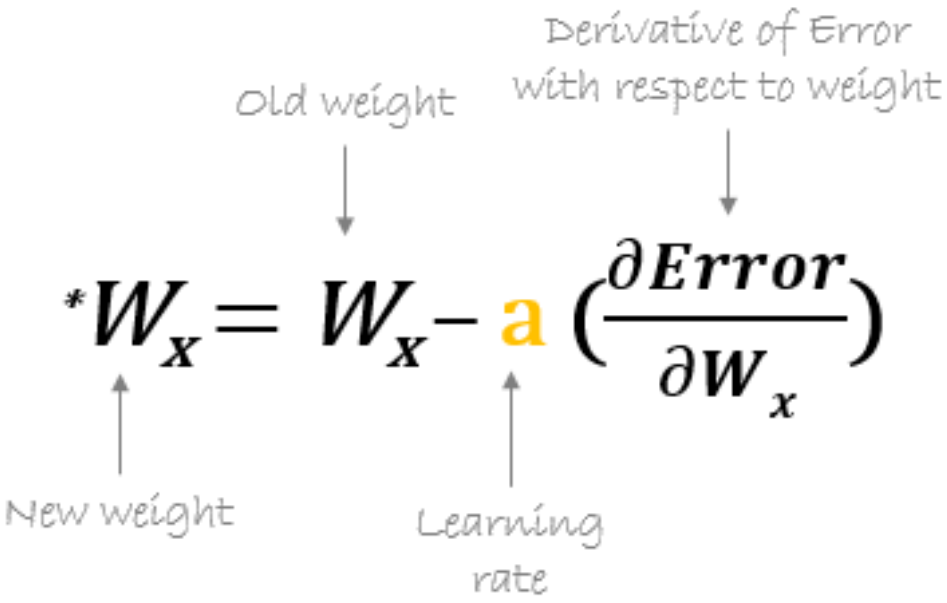
$$\text{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***

- The question now is how to change\update the weights value so that the error is reduced?
- The answer is Backpropagation!

Backpropagation

- Backpropagation, short for "backward propagation of errors", is a mechanism used to update the weights using gradient descent. It calculates the gradient of the error function with respect to the neural network's weights. The calculation proceeds backwards through the network.
- **Gradient descent** is an iterative optimization algorithm for finding the minimum of a function; in our case we want to minimize the error function. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.



- For example, to update w6, we take the current w6 and subtract the partial derivative of **error** function with respect to w6. Optionally, we multiply the derivative of the **error** function by a selected number to make sure that the new updated **weight** is minimizing the error function; this number is called ***learning rate***.

$$^{*}W_6 = W_6 - a\left(\frac{\partial Error}{\partial W_6}\right)$$

- The derivation of the error function is evaluated by applying the chain rule as following.

$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

chain rule

Error = $\frac{1}{2}$(prediction – actual)^2

prediction = $(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$

$$\frac{\partial Error}{\partial W_6} = \frac{\frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial Error}{\partial W_6} = (\mathbf{predictoin - actula}) * (h_2)$$

Δ = prediction – actual ← delta

$$\frac{\partial Error}{\partial W_6} = \Delta h_2$$

- So to update w6 we can apply the following formula.

$$^{*}W_6 = W_6 - a\,\Delta h_2$$

- Similarly, we can derive the update formula for w5 and any other weights existing between the output and the hidden layer.

$$^{*}W_5 = W_5 - a\,\Delta h_1$$

- However, when moving backward to update w1, w2, w3 and w4 existing between input and hidden layer, the partial derivative for the error function with respect to w1, for example, will be as following.

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$

chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

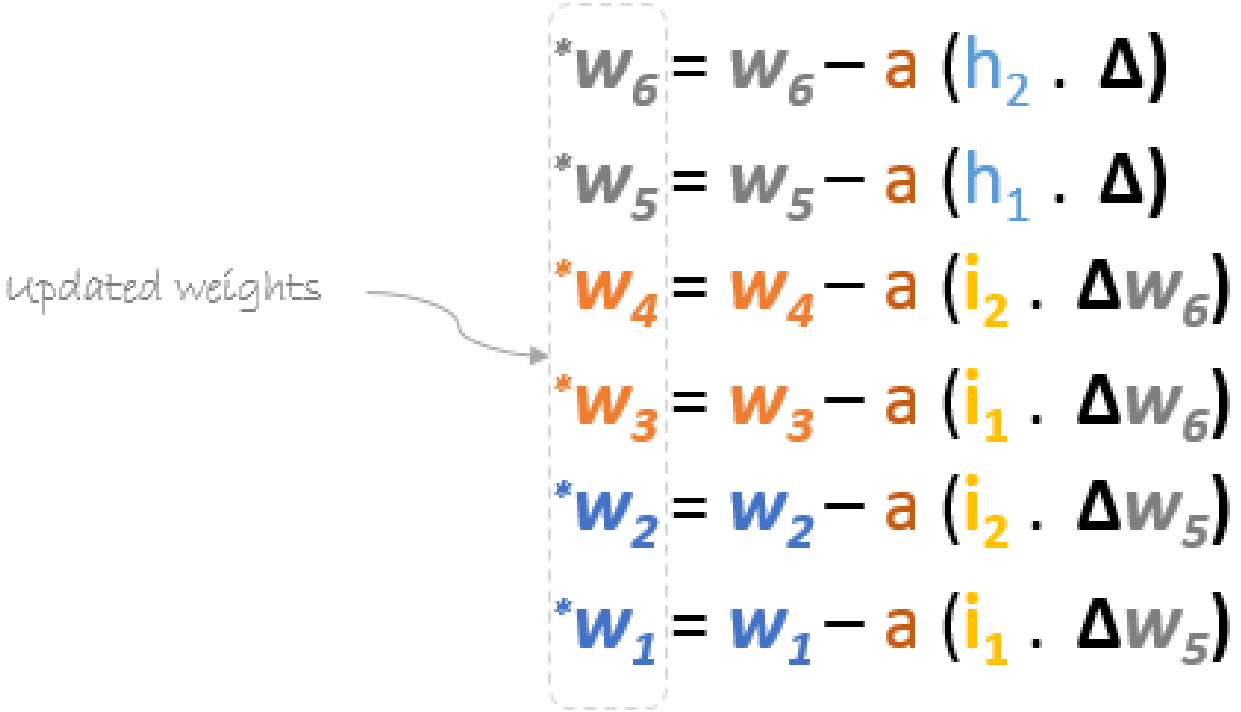
Error = $\frac{1}{2}$(prediction – actual)^2

prediction = $(h_1)\, w_5 + (h_2)\, w_6$

$h_1 = i_1 w_1 + i_2 w_2$

Δ = prediction – actual ← delta

- We can find the update formula for the remaining weights w2, w3 and w4 in the same way.
- In summary, the update formulas for all weights will be as following:



- We can rewrite the update formulas in matrices as following.

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . \begin{bmatrix} w_5 & w_6 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\, i_1 \Delta w_5 & a\, i_1 \Delta w_6 \\ a\, i_2 \Delta w_5 & a\, i_2 \Delta w_6 \end{bmatrix}$$

Backward Pass

- Using derived formulas we can find the new **weights**.
- **Learning rate:** is a hyperparameter which means that we need to manually guess its value.

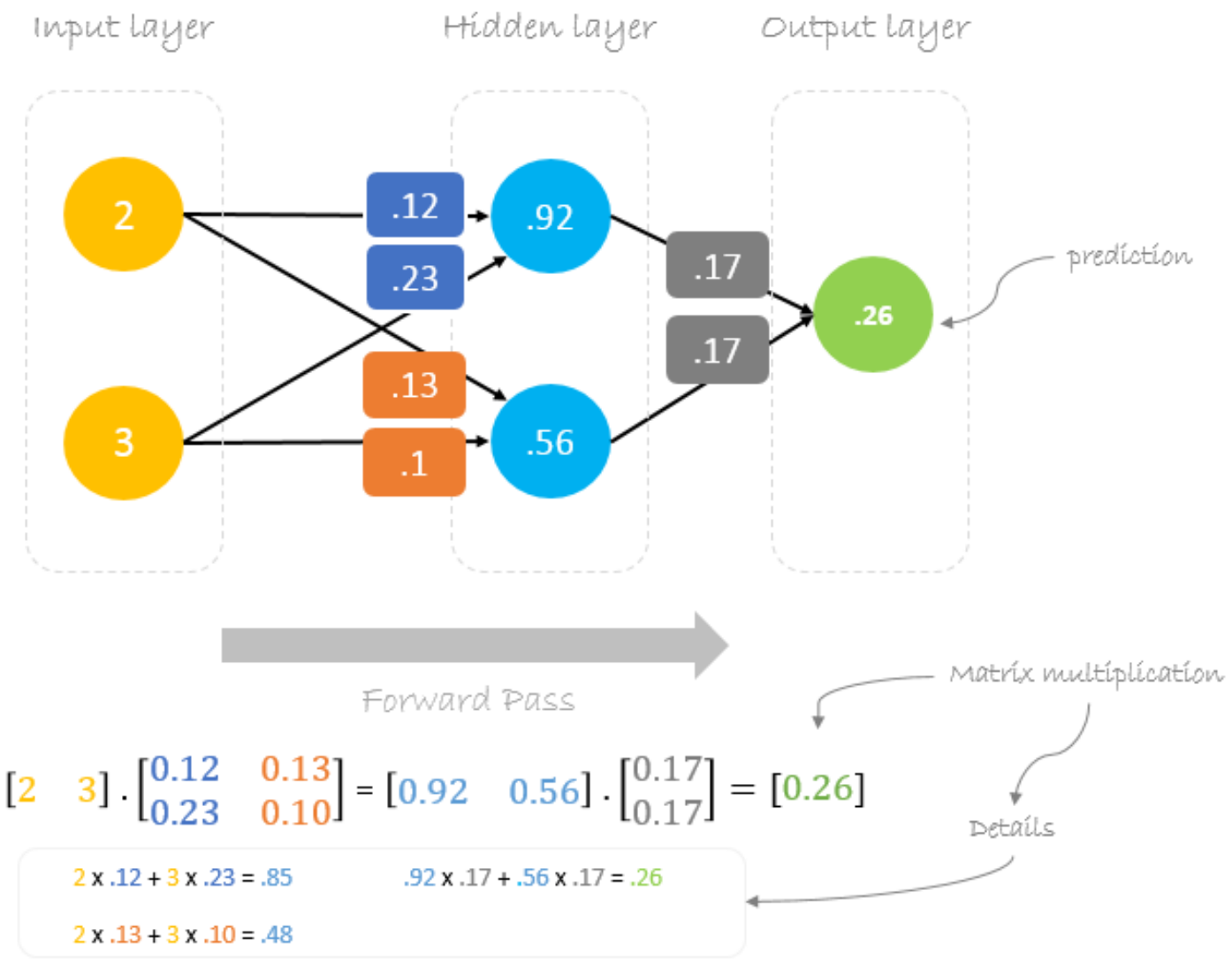
$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

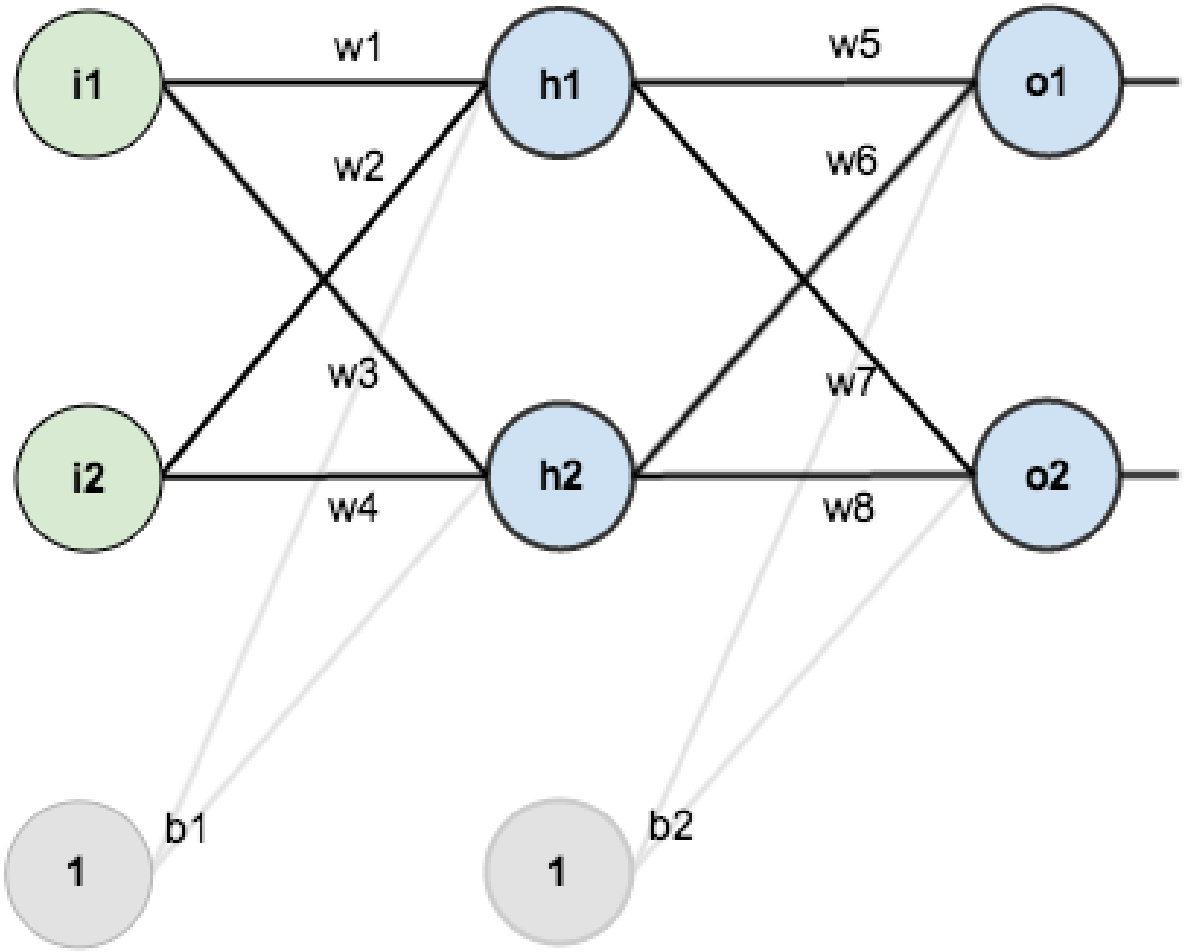
$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

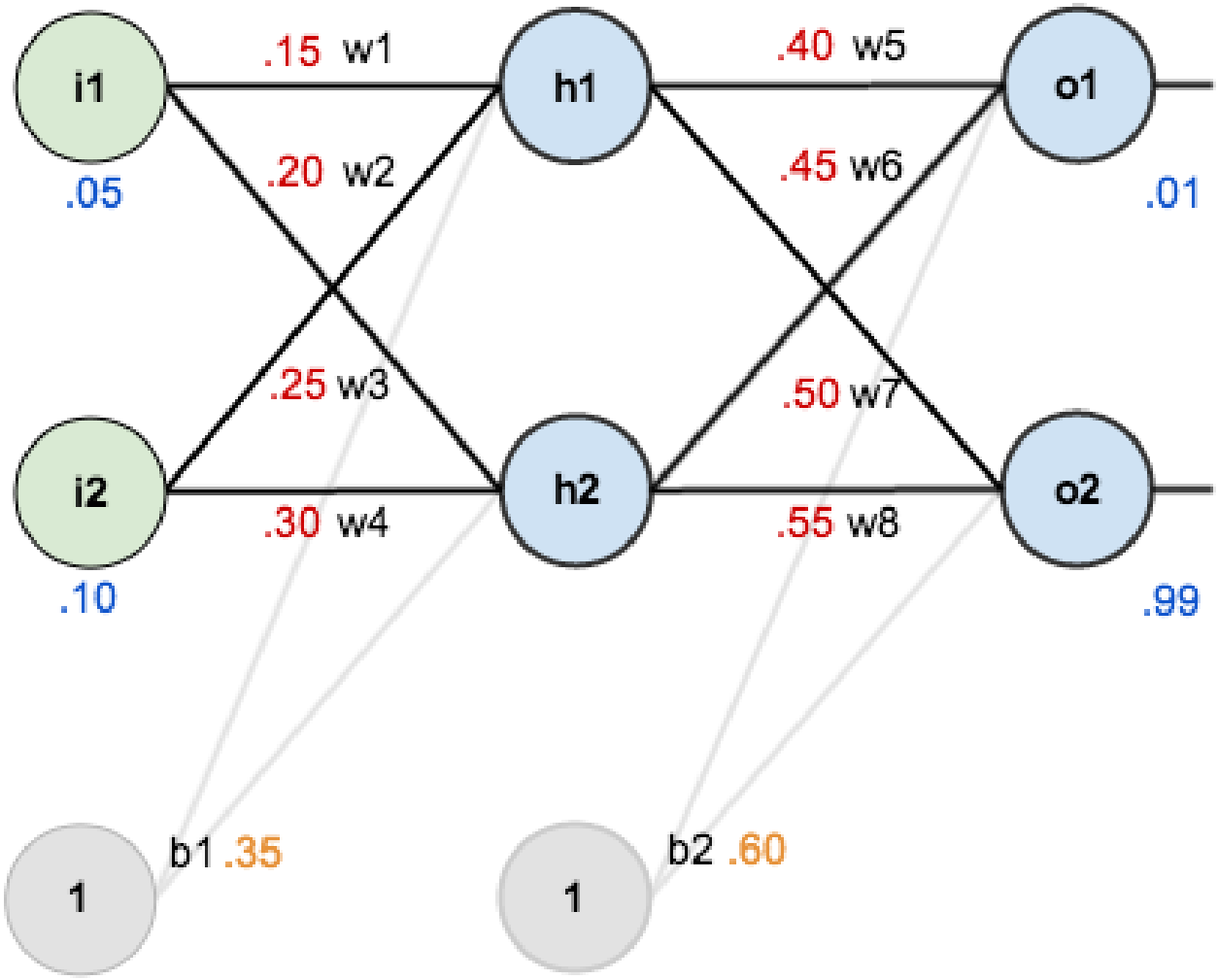
- Now, using the new **weights** we will repeat the forward passed.
- We can notice that the **prediction** 0.26 is a little bit closer to **actual output** than the previously predicted one 0.191. We can repeat the same process of backward and forward pass until **error** is close or equal to zero.



- For this tutorial, we're going to use a neural network with two inputs, two hidden neurons, two output neurons. Additionally, the hidden and output neurons will include a bias.
- Here's the basic structure:



- In order to have some numbers to work with, here are the initial weights, the biases, and training inputs/outputs:



- The goal of backpropagation is to optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs.
- For the rest of this tutorial we're going to work with a single training set: given inputs 0.05 and 0.10, we want the neural network to output 0.01 and 0.99.

The Forward Pass

- To begin, lets see what the neural network currently predicts given the weights and biases above and inputs of 0.05 and 0.10. To do this we'll feed those inputs forward though the network.
- We figure out the *total net input* to each hidden layer neuron, *squash* the total net input using an *activation function* (here we use the *logistic function*), then repeat the process with the output layer neurons.

- Here's how we calculate the total net input for h1:

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

- We then squash it using the logistic function to get the output of h1:

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

- Carrying out the same process for we get:

$$out_{h2} = 0.596884378$$

- We repeat this process for the output layer neurons, using the output from the hidden layer neurons as inputs.
- Here's the output for o1:

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

And carrying out the same process for o_2 we get:

$$out_{o2} = 0.772928465$$

- **Calculating the Total Error**
- We can now calculate the error for each output neuron using the squared error function and sum them to get the total error:

$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

- For example, the target output for is 0.01 but the neural network output 0.75136507, therefore its error is:

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

Repeating this process for o2 (remembering that the target is 0.99) we get:

$$E_{o2} = 0.023560026$$

- The total error for the neural network is the sum of these errors:

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

The Backwards Pass

- Our goal with backpropagation is to update each of the weights in the network so that they cause the actual output to be closer the target output, thereby minimizing the error for each output neuron and the network as a whole.

Output Layer

Consider w_5. We want to know how much a change in w_5 affects the total error, aka $\frac{\partial E_{total}}{\partial w_5}$.

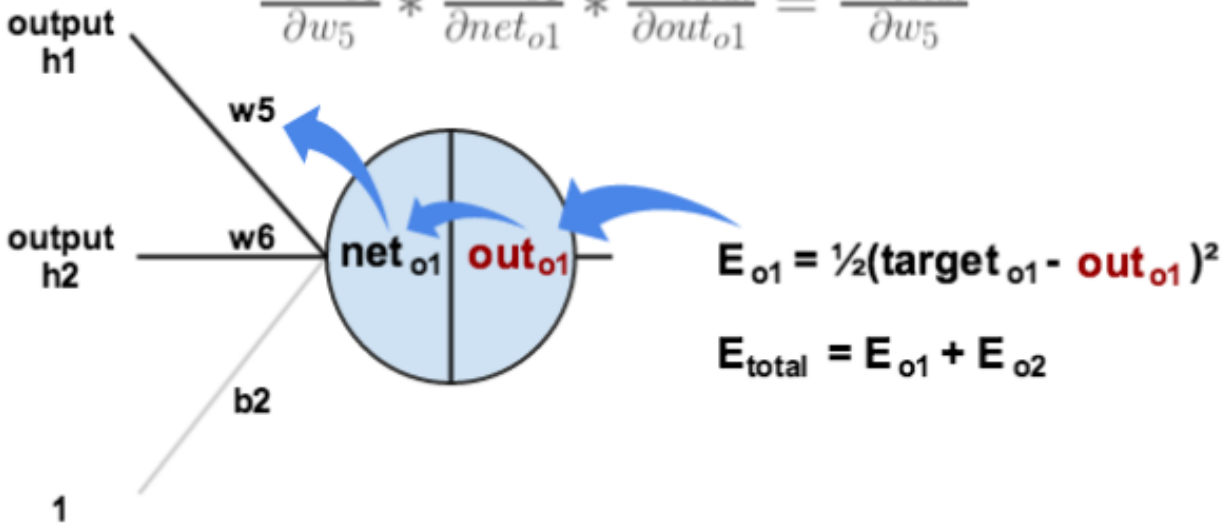
> $\frac{\partial E_{total}}{\partial w_5}$ is read as "the partial derivative of E_{total} with respect to w_5". You can also say "the gradient with respect to w_5".

By applying the chain rule we know that:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

Visually, here's what we're doing:

$$\frac{\partial net_{o1}}{\partial w_5} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5}$$



We need to figure out each piece in this equation.

First, how much does the total error change with respect to the output?

$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

$-(target - out)$ is sometimes expressed as $out - target$

When we take the partial derivative of the total error with respect to out_{o1}, the quantity $\frac{1}{2}(target_{o2} - out_{o2})^2$ becomes zero because out_{o1} does not affect it which means we're taking the derivative of a constant which is zero.

Next, how much does the output of o_1 change with respect to its total net input?

The partial derivative of the logistic function is the output multiplied by 1 minus the output:

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

Finally, how much does the total net input of $_{o}1$ change with respect to w_5?

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

Putting it all together:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

You'll often see this calculation combined in the form of the delta rule:

$$\frac{\partial E_{total}}{\partial w_5} = -(target_{o1} - out_{o1}) * out_{o1}(1 - out_{o1}) * out_{h1}$$

Alternatively, we have $\frac{\partial E_{total}}{\partial out_{o1}}$ and $\frac{\partial out_{o1}}{\partial net_{o1}}$ which can be written as $\frac{\partial E_{total}}{\partial net_{o1}}$, aka δ_{o1} (the Greek letter delta) aka the *node delta*. We can use this to rewrite the calculation above:

$$\delta_{o1} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} = \frac{\partial E_{total}}{\partial net_{o1}}$$

$$\delta_{o1} = -(target_{o1} - out_{o1}) * out_{o1}(1 - out_{o1})$$

Therefore:

$$\frac{\partial E_{total}}{\partial w_5} = \delta_{o1} out_{h1}$$

Some sources extract the negative sign from δ so it would be written as:

$$\frac{\partial E_{total}}{\partial w_5} = -\delta_{o1} out_{h1}$$

To decrease the error, we then subtract this value from the current weight (optionally multiplied by some learning rate, eta, which we'll set to 0.5):

$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

> Some sources use α (alpha) to represent the learning rate, others use η (eta), and others even use ϵ (epsilon).

We can repeat this process to get the new weights w_6, w_7, and w_8:

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

We perform the actual updates in the neural network *after* we have the new weights leading into the hidden layer neurons (ie, we use the original weights, not the updated weights, when we continue the backpropagation algorithm below).

Hidden Layer

Next, we'll continue the backwards pass by calculating new values for w_1, w_2, w_3, and w_4.

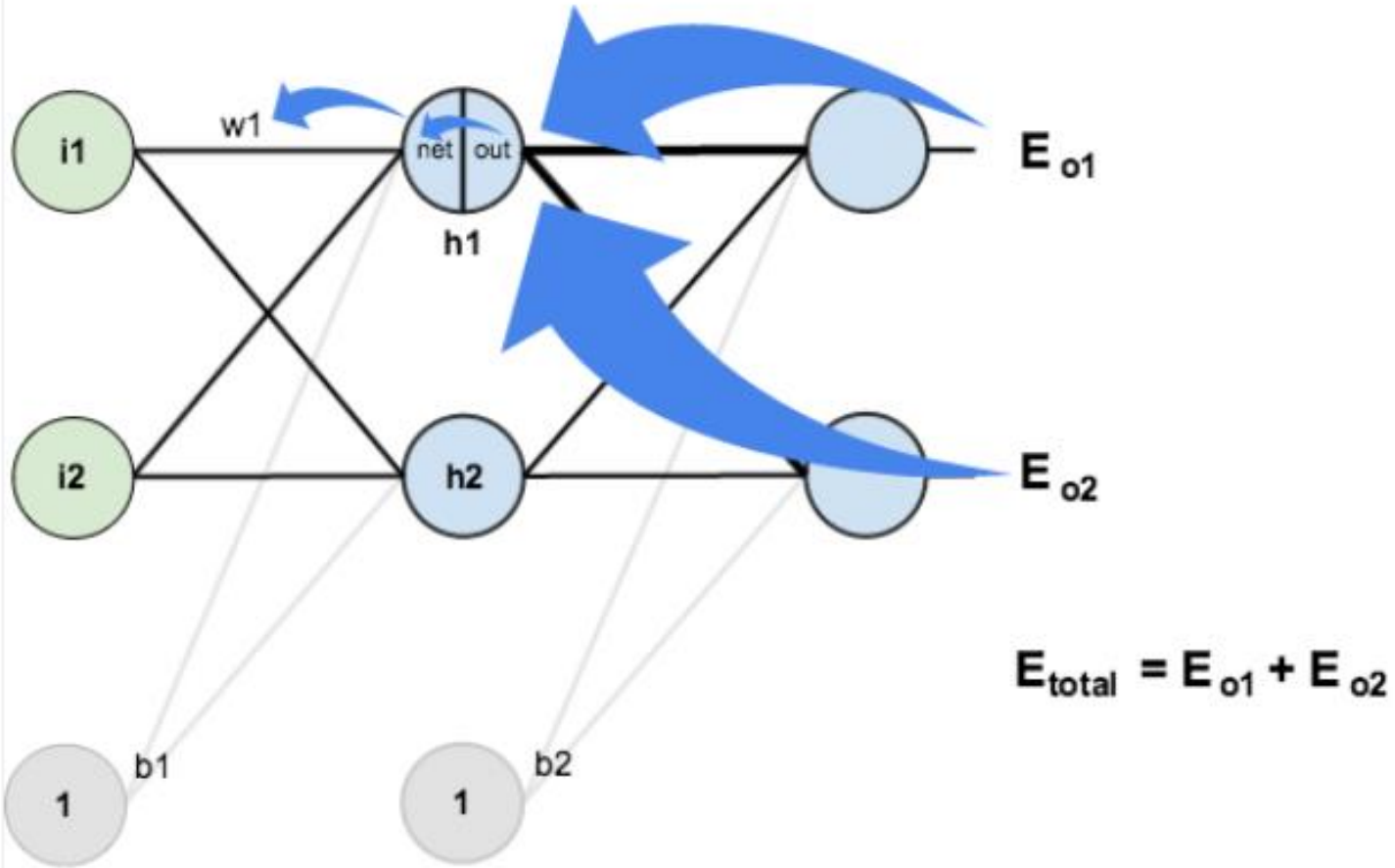
Big picture, here's what we need to figure out:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

Visually:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



We're going to use a similar process as we did for the output layer, but slightly different to account for the fact that the output of each hidden layer neuron contributes to the output (and therefore error) of multiple output neurons. We know that out_{h1} affects both out_{o1} and out_{o2} therefore the $\frac{\partial E_{total}}{\partial out_{h1}}$ needs to take into consideration its effect on the both output neurons:

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

Starting with $\frac{\partial E_{o1}}{\partial out_{h1}}$:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

We can calculate $\frac{\partial E_{o1}}{\partial net_{o1}}$ using values we calculated earlier:

$$\frac{\partial E_{o1}}{\partial net_{o1}} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} = 0.74136507 * 0.186815602 = 0.138498562$$

And $\frac{\partial net_{o1}}{\partial out_{h1}}$ is equal to w_5:

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial out_{h1}} = w_5 = 0.40$$

Plugging them in:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} = 0.138498562 * 0.40 = 0.055399425$$

Following the same process for $\frac{\partial E_{o2}}{\partial out_{h1}}$, we get:

$$\frac{\partial E_{o2}}{\partial out_{h1}} = -0.019049119$$

Therefore:

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} = 0.055399425 + -0.019049119 = 0.036350306$$

Now that we have $\frac{\partial E_{total}}{\partial out_{h1}}$, we need to figure out $\frac{\partial out_{h1}}{\partial net_{h1}}$ and then $\frac{\partial net_{h1}}{\partial w}$ for each weight:

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}}$$

$$\frac{\partial out_{h1}}{\partial net_{h1}} = out_{h1}(1 - out_{h1}) = 0.59326999(1 - 0.59326999) = 0.241300709$$

Putting it all together:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

You might also see this written as:

$$\frac{\partial E_{total}}{\partial w_1} = (\sum_o \frac{\partial E_{total}}{\partial out_o} * \frac{\partial out_o}{\partial net_o} * \frac{\partial net_o}{\partial out_{h1}}) * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = (\sum_o \delta_o * w_{ho}) * out_{h1}(1 - out_{h1}) * i_1$$

$$\frac{\partial E_{total}}{\partial w_1} = \delta_{h1} i_1$$

We can now update w_1:

$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

Repeating this for w_2, w_3, and w_4

$$w_2^+ = 0.19956143$$

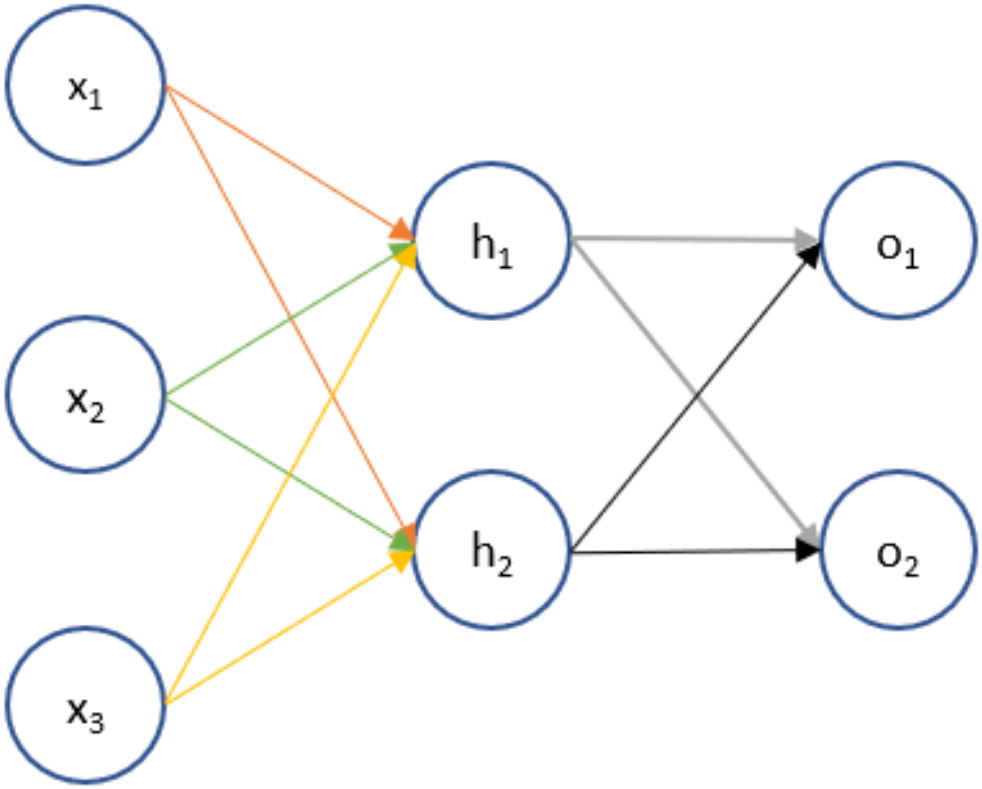
$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

Finally, we've updated all of our weights! When we fed forward the 0.05 and 0.1 inputs originally, the error on the network was 0.298371109. After this first round of backpropagation, the total error is now down to 0.291027924. It might not seem like much, but after repeating this process 10,000 times, for example, the error plummets to 0.0000351085. At this point, when we feed forward 0.05 and 0.1, the two outputs neurons generate 0.015912196 (vs 0.01 target) and 0.984065734 (vs 0.99 target).

Example 2:

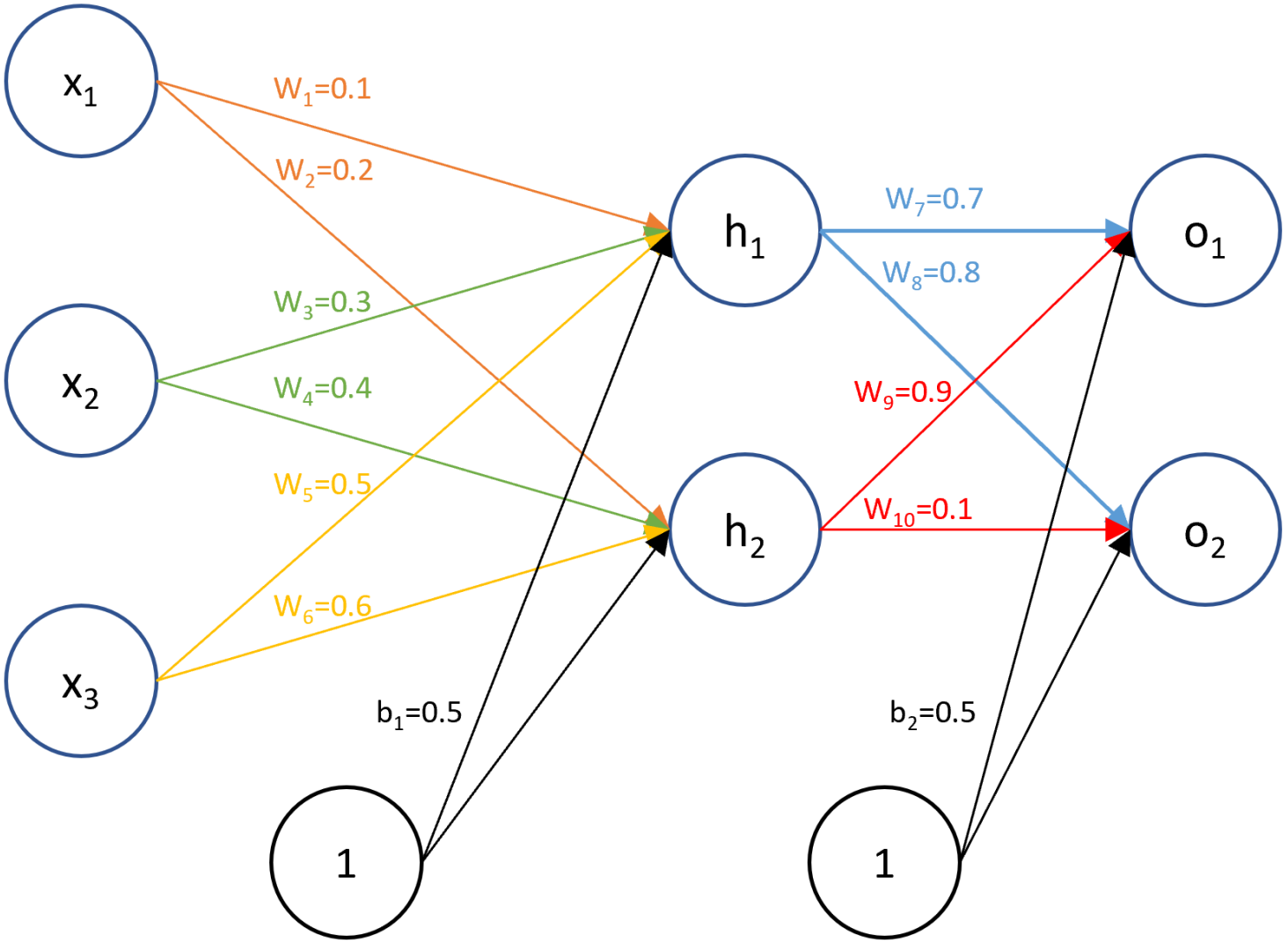
- The neural network I use has three input neurons, one hidden layer with two neurons, and an output layer with two neurons.



Example 2:

- The following are the (very) high level steps that I will take in this example. Details on each step will follow after.
- (1) Initialize weights for the parameters we want to train
- (2) Forward propagate through the network to get the output values
- (3) Define the error or cost function and its first derivatives
- (4) Backpropagate through the network to determine the error derivatives
- (5) Update the parameter estimates using the error derivative and the current value

The input and target values for this problem are $x_1 = 1, x_2 = 4, x_3 = 5$ and $t_1 = 0.1, t_2 = 0.05$. I will initialize weights as shown in the diagram below. Generally, you will assign them randomly but for illustration purposes, I've chosen these numbers.



Mathematically, we have the following relationships between nodes in the networks. For the input and output layer, I will use the somewhat strange convention of denoting z_{h_1}, z_{h_2}, z_{o_1}, and z_{o_2} to denote the value before the activation function is applied and the notation of h_1, h_2, o_1, and o_2 to denote the values after application of the activation function.

Input to hidden layer

$$w_1 x_1 + w_3 x_2 + w_5 x_3 + b_1 = z_{h_1}$$

$$w_2 x_1 + w_4 x_2 + w_6 x_3 + b_1 = z_{h_2}$$

$$h_1 = \sigma(z_{h_1})$$

$$h_2 = \sigma(z_{h_2})$$

Hidden layer to output layer

$$w_7 h_1 + w_9 h_2 + b_2 = z_{o_1}$$

$$w_8 h_1 + w_{10} h_2 + b_2 = z_{o_2}$$

$$o_1 = \sigma(z_{o_1})$$

$$o_2 = \sigma(z_{o_2})$$

We can use the formulas above to forward propagate through the network. I've shown up to four decimal places below but maintained all decimals in actual calculations.

$$w_1x_1 + w_3x_2 + w_5x_3 + b_1 = z_{h_1} = 0.1(1) + 0.3(4) + 0.5(5) + 0.5 = 4.3$$
$$h_1 = \sigma(z_{h_1}) = \sigma(4.3) = 0.9866$$

$$w_2x_1 + w_4x_2 + w_6x_3 + b_1 = z_{h_2} = 0.2(1) + 0.4(4) + 0.6(5) + 0.5 = 5.3$$
$$h_2 = \sigma(z_{h_2}) = \sigma(5.3) = 0.9950$$

$$w_7h_1 + w_9h_2 + b_2 = z_{o_1} = 0.7(0.9866) + 0.9(0.9950) + 0.5 = 2.0862$$
$$o_1 = \sigma(z_{o_1}) = \sigma(2.0862) = 0.8896$$

$$w_8h_1 + w_{10}h_2 + b_2 = z_{o_2} = 0.8(0.9866) + 0.1(0.9950) + 0.5 = 1.3888$$
$$o_2 = \sigma(z_{o_2}) = \sigma(1.3888) = 0.8004$$

We now define the sum of squares error using the target values and the results from the last layer from forward propagation.

$$E = \frac{1}{2}[(o_1 - t_1)^2 + (o_2 - t_2)^2$$

$$\frac{dE}{d_{o_1}} = o_1 - t_1$$

$$\frac{dE}{d_{o_2}} = o_2 - t_2$$

First we go over some derivatives we will need in this step. The derivative of the sigmoid function is given here. Also, given that $w_7h_1 + w_9h_2 + b_2 = z_{o_1}$ and $w_8h_1 + w_{10}h_2 + b_2 = z_{o_2}$, we have $\frac{dz_{o_1}}{dw_7} = h_1$, $\frac{dz_{o_2}}{dw_8} = h_1$, $\frac{dz_{o_1}}{dw_9} = h_2$, $\frac{dz_{o_2}}{dw_{10}} = h_2$, $\frac{dz_{o_1}}{db_2} = 1$, and $\frac{dz_{o_2}}{db_2} = 1$.

The sigmoid activation function is given by

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\frac{d\sigma}{dx} = \frac{e^{-x}}{(1+e^{-x})^2}$$

We can write $\frac{e^{-x}}{1+e^{-x}}$ as $1 - \frac{1}{1+e^{-x}}$. The derivative can be written as

$$\frac{d\sigma}{dx} = \frac{1}{1+e^{-x}}(1 - \sigma(x))$$

$$\frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x))$$

We are now ready to calculate $\frac{dE}{dw_7}$, $\frac{dE}{dw_8}$, $\frac{dE}{dw_9}$, and $\frac{dE}{dw_{10}}$ using the derivatives we have already discussed.

$$\frac{dE}{dw_7} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{dw_7}$$

$$\frac{dE}{dw_7} = (o_1 - t_1)(o_1(1 - o_1))h_1$$

$$\frac{dE}{dw_7} = (0.8896 - 0.1)(0.8896(1 - 0.8896))(0.9866)$$

$$\frac{dE}{dw_7} = 0.0765$$

$$\frac{dE}{dw_8} = \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{dw_8}$$

$$\frac{dE}{dw_8} = (0.7504)(0.1598)(0.9866)$$

$$\frac{dE}{dw_8} = 0.1183$$

$$\frac{dE}{dw_9} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{dw_9}$$

$$\frac{dE}{dw_9} = (0.7896)(0.0983)(0.9950)$$

$$\frac{dE}{dw_9} = 0.0772$$

$$\frac{dE}{dw_{10}} = \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{dw_{10}}$$

$$\frac{dE}{dw_{10}} = (0.7504)(0.1598)(0.9950)$$

$$\frac{dE}{dw_{10}} = 0.1193$$

The error derivative of b_2 is a little bit more involved since changes to b_2 affect the error through both o_1 and o_2.

$$\frac{dE}{db_2} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{db_2} + \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{db_2}$$

$$\frac{dE}{db_2} = (0.7896)(0.0983)(1) + (0.7504)(0.1598)(1)$$

$$\frac{dE}{db_2} = 0.1975$$

To summarize, we have computed numerical values for the error derivatives with respect to w_7, w_8, w_9, w_{10}, and b_2. We will now backpropagate one layer to compute the error derivatives of the parameters connecting the input layer to the hidden layer. These error derivatives are $\frac{dE}{dw_1}$, $\frac{dE}{dw_2}$, $\frac{dE}{dw_3}$, $\frac{dE}{dw_4}$, $\frac{dE}{dw_5}$, $\frac{dE}{dw_6}$, and $\frac{dE}{db_1}$.

I will calculate $\frac{dE}{dw_1}$, $\frac{dE}{dw_3}$, and $\frac{dE}{dw_5}$ first since they all flow through the h_1 node.

$$\frac{dE}{dw_1} = \frac{dE}{dh_1}\frac{dh_1}{dz_{h_1}}\frac{dz_{h_1}}{dw_1}$$

The calculation of the first term on the right hand side of the equation above is a bit more involved than previous calculations since h_1 affects the error through both o_1 and o_2.

$$\frac{dE}{dh_1} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{dh_1} + \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{dh_1}$$

$$\frac{dE}{dh_1} = (0.7896)(0.0983)(0.7) + (0.7504)(0.1598)(0.8) = 0.1502$$

Plugging the above into the formula for $\frac{dE}{dw_1}$, we get

$$\frac{dE}{dw_1} = (0.1502)(0.0132)(1) = 0.0020$$

The calculations for $\frac{dE}{dw_3}$ and $\frac{dE}{dw_5}$ are below

$$\frac{dE}{dw_3} = \frac{dE}{dh_1}\frac{dh_1}{dz_{h_1}}\frac{dz_{h_1}}{dw_3}$$

$$\frac{dE}{dw_3} = (0.1502)(0.0132)(4) = 0.0079$$

$$\frac{dE}{dw_5} = \frac{dE}{dh_1}\frac{dh_1}{dz_{h_1}}\frac{dz_{h_1}}{dw_5}$$

$$\frac{dE}{dw_5} = (0.1502)(0.0132)(5) = 0.0099$$

I will now calculate $\frac{dE}{dw_2}$, $\frac{dE}{dw_4}$, and $\frac{dE}{dw_6}$ since they all flow through the h_2 node.

$$\frac{dE}{dw_2} = \frac{dE}{dh_2}\frac{dh_2}{dz_{h_2}}\frac{dz_{h_2}}{dw_2}$$

The calculation of the first term on the right hand side of the equation above is a bit more involved since h_2 affects the error through both o_1 and o_2.

$$\frac{dE}{dh_2} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{dh_2} + \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{dh_2}$$

$$\frac{dE}{dh_2} = (0.7896)(0.0983)(0.9) + (0.7504)(0.1598)(0.1) = 0.0818$$

Plugging the above into the formula for $\frac{dE}{dw_2}$, we get

$$\frac{dE}{dw_2} = (0.0818)(0.0049)(1) = 0.0004$$

The calculations for $\frac{dE}{dw_4}$ and $\frac{dE}{dw_6}$ are below

$$\frac{dE}{dw_4} = \frac{dE}{dh_2}\frac{dh_2}{dz_{h_2}}\frac{dz_{h_2}}{dw_4}$$

$$\frac{dE}{dw_4} = (0.0818)(0.0049)(4) = 0.0016$$

$$\frac{dE}{dw_6} = \frac{dE}{dh_2}\frac{dh_2}{dz_{h_2}}\frac{dz_{h_2}}{dw_6}$$

$$\frac{dE}{dw_6} = (0.0818)(0.0049)(5) = 0.0020$$

The final error derivative we have to calculate is $\frac{dE}{db_1}$, which is done next

$$\frac{dE}{db_1} = \frac{dE}{do_1}\frac{do_1}{dz_{o_1}}\frac{dz_{o_1}}{dh_1}\frac{dh_1}{dz_{h_1}}\frac{dz_{h_1}}{db_1} + \frac{dE}{do_2}\frac{do_2}{dz_{o_2}}\frac{dz_{o_2}}{dh_2}\frac{dh_2}{dz_{h_2}}\frac{dz_{h_2}}{db_1}$$

$$\frac{dE}{db_1} = (0.7896)(0.0983)(0.7)(0.0132)(1)+(0.7504)(0.1598)(0.1)(0.0049)(1) = 0.0008$$

We now have all the error derivatives and we're ready to make the parameter updates after the first iteration of backpropagation. We will use the learning rate of $\alpha = 0.01$

$$w_1 := w_1 - \alpha \frac{dE}{dw_1} = 0.1 - (0.01)(0.0020) = 0.1000$$

$$w_2 := w_2 - \alpha \frac{dE}{dw_2} = 0.2 - (0.01)(0.0004) = 0.2000$$

$$w_3 := w_3 - \alpha \frac{dE}{dw_3} = 0.3 - (0.01)(0.0079) = 0.2999$$

$$w_4 := w_4 - \alpha \frac{dE}{dw_4} = 0.4 - (0.01)(0.0016) = 0.4000$$

$$w_5 := w_5 - \alpha \frac{dE}{dw_5} = 0.5 - (0.01)(0.0099) = 0.4999$$

$$w_6 := w_6 - \alpha \frac{dE}{dw_6} = 0.6 - (0.01)(0.0020) = 0.6000$$

$$w_7 := w_7 - \alpha \frac{dE}{dw_7} = 0.7 - (0.01)(0.0765) = 0.6992$$

$$w_8 := w_8 - \alpha \frac{dE}{dw_8} = 0.8 - (0.01)(0.1183) = 0.7988$$

$$w_9 := w_9 - \alpha \frac{dE}{dw_9} = 0.9 - (0.01)(0.0772) = 0.8992$$

$$w_{10} := w_{10} - \alpha \frac{dE}{dw_{10}} = 0.1 - (0.01)(0.1193) = 0.0988$$

$$b_1 := b_1 - \alpha \frac{dE}{db_1} = 0.5 - (0.01)(0.0008) = 0.5000$$

$$b_2 := b_2 - \alpha \frac{dE}{db_2} = 0.5 - (0.01)(0.1975) = 0.4980$$

So what do we do now? We repeat that over and over many times until the error goes down and the parameter estimates stabilize or converge to some values. We obviously won't be going through all these calculations manually. I've provided Python code below that codifies the calculations above. Nowadays, we wouldn't do any of these manually but rather use a machine learning package that is already readily available.

In [1]:
```
import numpy as np
import pandas as pd
%matplotlib inline
```

In [2]:
```
numIter = 10000

# Initialize Variables
w1 = 0.1
w2 = 0.2
w3 = 0.3
w4 = 0.4
w5 = 0.5
w6 = 0.6
w7 = 0.7
w8 = 0.8
w9 = 0.9
w10 = 0.1
wList = [w1, w2, w3, w4, w5, w6, w7, w8, w9, w10]

b1 = 0.5
b2 = 0.5
bList = [b1, b2]

# Input and Target values
x1 = 1
x2 = 4
x3 = 5
xList = [x1, x2, x3]

t1 = 0.1
t2 = 0.05
tList = [t1, t2]

# set learning rate
alpha = 0.01
```

In [3]:
```
def sigmoid(x):
    return np.divide(1, (1 + np.exp(-x)))
```

In [4]:
```
def forwardProp(xList, wList, bList):
    zh1 = wList[0] * xList[0] + wList[2] * xList[1] + wList[4] * xList[2] + bList[0]
    zh2 = wList[1] * xList[0] + wList[3] * xList[1] + wList[5] * xList[2] + bList[0]
    h1 = sigmoid(zh1)
    h2 = sigmoid(zh2)
    zo1 = wList[6] * h1 + wList[8] * h2 + bList[1]
    zo2 = wList[7] * h1 + wList[9] * h2 + bList[1]
    o1 = sigmoid(zo1)
    o2 = sigmoid(zo2)
    return h1, h2, o1, o2
```

In [5]:
```
def error(oList, tList):
    return 0.5 * (np.power(oList[0] - tList[0], 2) + np.power(oList[1] - tList[1], 2))
```

In [6]:

```
errList = []
for i in range(numIter):
    # Forward propagation
    h1, h2, o1, o2 = forwardProp(xList, wList, bList)

    # Compute Error
    sse = error([o1, o2], tList)
    errList.append(sse)

    print('Running ' + str(i + 1) + ' of ' + str(numIter))
    print('o1: ' + str(o1))
    print('t1: ' + str(t1))
    print('o2: ' + str(o2))
    print('t2: ' + str(t2))
    print('error: ' + str(sse))
    print('')

    # Error derivative calculations
    # Compute dE_dw7
    dE_do1 = o1 - t1
    do1_dzo1 = o1 * (1-o1)
    dzo1_dw7 = h1
    dE_dw7 = dE_do1 * do1_dzo1 * dzo1_dw7
    # Compute dE_dw8
    dE_do2 = o2 - t2
    do2_dzo2 = o2 * (1 - o2)
    dzo2_dw8 = h1
    dE_dw8 = dE_do2 * do2_dzo2 * dzo2_dw8
    # Compute dE_dw9
    dzo1_dw9 = h2
    dE_dw9 = dE_do1 * do1_dzo1 * dzo1_dw9
    # Compute dE_dw10
    dzo2_dw10 = h2
    dE_dw10 = dE_do2 * do2_dzo2 * dzo2_dw10
```

```
# Compute dE_db2
dzo1_db2 = 1
dzo2_db2 = 1
dE_db2 = dE_do1 * do1_dzo1 * dzo1_db2 + dE_do2 * do2_dzo2 * dzo2_db2
# Compute dE_dh1 first
dzo1_dh1 = w7
dzo2_dh1 = w8
dE_dh1 = dE_do1 * do1_dzo1 * dzo1_dh1 + dE_do2 * do2_dzo2 * dzo2_dh1
# Compute dE_dw1
dh1_dzh1 = h1 * (1 - h1)
dzh1_dw1 = x1
dE_dw1 = dE_dh1 * dh1_dzh1 * dzh1_dw1
# Compute dE_dw3
dzh1_dw3 = x2
dE_dw3 = dE_dh1 * dh1_dzh1 * dzh1_dw3
# Compute dE_dw5
dzh1_dw5 = x3
dE_dw5 = dE_dh1 * dh1_dzh1 * dzh1_dw5
# Compute dE_dh2 first
dzo1_dh2 = w9
dzo2_dh2 = w10
dE_dh2 = dE_do1 * do1_dzo1 * dzo1_dh2 + dE_do2 * do2_dzo2 * dzo2_dh2
# Compute dE_dw2
dh2_dzh2 = h2 * (1 - h2)
dzh2_dw2 = x1
dE_dw2 = dE_dh2 * dh2_dzh2 * dzh2_dw2
# Compute dE_dw4
dzh2_dw4 = x2
dE_dw4 = dE_dh2 * dh2_dzh2 * dzh2_dw4
# Compute dE_dw6
dzh2_dw6 = x3
dE_dw6 = dE_dh2 * dh2_dzh2 * dzh2_dw6
# Compute dE_db1
dzh1_db1 = 1
dzh2_db1 = 1
term1 = dE_do1 * do1_dzo1 * dzo1_dh1 * dh1_dzh1 * dzh1_db1
term2 = dE_do2 * do2_dzo2 * dzo2_dh2 * dh2_dzh2 * dzh2_db1
dE_db1 = term1 + term2
```

```
    # Update all parameters
    w1 = w1 - alpha * dE_dw1
    w2 = w2 - alpha * dE_dw2
    w3 = w3 - alpha * dE_dw3
    w4 = w4 - alpha * dE_dw4
    w5 = w5 - alpha * dE_dw5
    w6 = w6 - alpha * dE_dw6
    w7 = w7 - alpha * dE_dw7
    w8 = w8 - alpha * dE_dw8
    w9 = w9 - alpha * dE_dw9
    w10 = w10 - alpha * dE_dw10
    b1 = b1 - alpha * dE_db1
    b2 = b2 - alpha * dE_db2
    wList = [w1, w2, w3, w4, w5, w6, w7, w8, w9, w10]
    bList = [b1, b2]
```

I ran 10,000 iterations and we see below that sum of squares error has dropped significantly after the first thousand or so iterations.

```
In [7]: pd.DataFrame(errList, columns=['SSE']).plot()
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0x1b6cad97668>
```